

Lecture 4 Summary: Policy Gradient Methods

1. Motivation

Value-based methods (e.g., Value Iteration, Q-Learning) learn value functions and then act greedily. However, they can become unstable or inefficient with nonlinear function approximation. Policy-gradient methods instead *directly* parameterize and optimize the policy itself:

$$J(\theta) = \mathbb{E}_{\pi_\theta} \left[\sum_{t=0}^{\infty} \gamma^t r_t \right].$$

The goal is to find parameters θ that maximize $J(\theta)$.

2. Stochastic Policy Parameterizations

Setting	Example	Log-gradient
Discrete actions	$\pi_\theta(a \mid s) = \frac{e^{f_\theta(s,a)}}{\sum_{a'} e^{f_\theta(s,a')}}$	$\nabla_\theta \log \pi_\theta(a \mid s) = \nabla_\theta f_\theta(s, a) - \sum_{a'} \pi_\theta(a' \mid s) \nabla_\theta f_\theta(s, a')$
Continuous actions	Gaussian $\mathcal{N}(\mu_\theta(s), \sigma^2)$	$\nabla_\theta \log \pi_\theta(a \mid s) = \frac{a - \mu_\theta(s)}{\sigma^2} \nabla_\theta \mu_\theta(s)$

Table 1: Common stochastic policy parameterizations and their log-gradients.

These forms make $\nabla_\theta \log \pi_\theta(a \mid s)$ directly computable by autodiff.

3. Policy Gradient Theorem

The *Policy Gradient Theorem* gives a simple, model-free expression for the gradient:

$$\nabla_\theta J(\theta) = \mathbb{E}_{s \sim d^{\pi_\theta}, a \sim \pi_\theta} \left[Q^{\pi_\theta}(s, a) \nabla_\theta \log \pi_\theta(a \mid s) \right]. \quad (1)$$

Key insight: the derivative does not include $\nabla_{\theta} d^{\pi_{\theta}}(s)$, so we can estimate this gradient using on-policy samples.

4. REINFORCE Algorithm (Williams, 1992)

A Monte Carlo estimator of the gradient:

$$\hat{g} = \sum_{t=0}^{T-1} R_t \nabla_{\theta} \log \pi_{\theta}(a_t \mid s_t), \quad R_t = \sum_{t' \geq t} \gamma^{t'-t} r_{t'}.$$

Algorithm 1 REINFORCE

```

0: Initialize policy parameters  $\theta$ 
0: for each episode do
0:   Sample trajectory  $\tau = (s_0, a_0, r_0, \dots, s_T)$  under  $\pi_{\theta}$ 
0:   for each timestep  $t$  in  $\tau$  do
0:     Compute  $R_t = \sum_{t' \geq t} \gamma^{t'-t} r_{t'}$ 
0:      $\theta \leftarrow \theta + \alpha R_t \nabla_{\theta} \log \pi_{\theta}(a_t \mid s_t)$ 
0:   end for
0: end for

```

REINFORCE is unbiased but has high variance.

5. Variance Reduction via Baselines

Subtracting a baseline $b(s)$ that does not depend on a leaves the gradient unbiased:

$$\nabla_{\theta} J(\theta) = \mathbb{E}[(Q^{\pi}(s, a) - b(s)) \nabla_{\theta} \log \pi_{\theta}(a \mid s)].$$

Choosing $b(s) = V^{\pi}(s)$ minimizes variance and defines the *Advantage*:

$$A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s).$$

The update then becomes:

$$\theta \leftarrow \theta + \alpha A_t \nabla_{\theta} \log \pi_{\theta}(a_t \mid s_t).$$

6. Actor–Critic Methods

In practice, Q^π or V^π are unknown and learned with a *critic*. The critic minimizes TD-error:

$$\delta_t = r_t + \gamma V_\omega(s_{t+1}) - V_\omega(s_t), \quad \omega \leftarrow \omega - \beta \delta_t \nabla_\omega V_\omega(s_t).$$

The actor then updates using δ_t as a low-variance estimate of the advantage:

$$\theta \leftarrow \theta + \alpha \delta_t \nabla_\theta \log \pi_\theta(a_t \mid s_t).$$

7. Average Reward (Undiscounted) Case

For continuing tasks, the objective is the average steady-state reward:

$$\rho^\pi = \sum_s d^\pi(s) \sum_a \pi(a \mid s) R(s, a),$$

and the gradient takes the same form:

$$\nabla_\theta \rho^\pi = \mathbb{E}_{s,a \sim \pi} [A^\pi(s, a) \nabla_\theta \log \pi_\theta(a \mid s)].$$

8. Conservative and Trust-Region Policy Updates

Large policy changes can degrade performance. To guarantee monotonic improvement:

$$\pi_{k+1} = (1 - \alpha) \pi_k + \alpha \pi_k^*,$$

or impose a KL constraint (as in TRPO):

$$\begin{aligned} \max_{\theta} \quad & \mathbb{E} \left[\frac{\pi_\theta(a \mid s)}{\pi_{\theta_{\text{old}}}(a \mid s)} A^{\pi_{\theta_{\text{old}}}}(s, a) \right] \\ \text{s.t.} \quad & \mathbb{E} [\text{KL}(\pi_{\theta_{\text{old}}} \parallel \pi_\theta)] \leq \delta. \end{aligned} \tag{2}$$

The practical approximation is PPO’s *clipped surrogate* objective.

9. Key Takeaways

- Policy Gradient Theorem enables model-free differentiation of expected return.

- REINFORCE is unbiased but high-variance; baselines reduce variance.
- Actor–Critic uses bootstrapped value estimates to stabilize learning.
- TRPO/PPO constrain policy updates for reliable improvement.
- Policy-based methods handle continuous actions and stochasticity naturally.

In one line: Policy gradient methods learn by differentiating through the policy itself; REINFORCE provides the estimator, baselines and critics control variance, and trust-region methods make deep RL stable.